

Security Findings Report

FalconStrix – Host Intrusion Detection & Response Dashboard

This report documents two related vulnerabilities discovered during a self-directed web security assessment of the FalconStrix project, performed as hands-on practice applying concepts from TryHackMe and PortSwigger's Web Security Academy. Both findings were identified through black-box testing with Burp Suite, confirmed via source-code review, remediated, and independently re-verified.

Report ID	FALSTRX-2026-002
Target Application	FalconStrix SOC Dashboard (Flask / Socket.IO / MySQL)
Findings Covered	FALSTRX-2026-001, FALSTRX-2026-003
Discovered By	Manual black-box testing (Burp Suite) + source-code review
Environment	Isolated local lab – Windows 11 target, Kali Linux (VMware) attacker
Assessment Dates	22 – 23 July 2026
Overall Status	Both findings remediated and verified

ID	Title	Class (CWE)	Severity	Status
FALSTRX-2026-001	Unauthenticated debug endpoints expose internal telemetry	CWE-906	High	Fixed & Verified
FALSTRX-2026-003	Auth lockout silently auto-bypassed; unlock endpoint missing role checks	CWE-287 / CWE-400	Medium	Fixed & Verified

Finding 1 — Unauthenticated Debug Endpoints (CWE-306)

Report ID	FALSTRX-2026-001
Affected Component	gui_dashboard/app.py – /api/debug/* endpoints
Vulnerability Class	Missing Authentication for Critical Function (CWE-306)
Severity	High — CVSS 3.1: 7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
Status	Fixed & Verified — 22 July 2026

1.1 Summary

FalconStrix exposed internal debug endpoints (/api/debug/events and /api/debug/last-snapshot) that were explicitly excluded from the application's authentication check. Any unauthenticated client with network access could retrieve live internal telemetry — system state, active alert counts, process-kill counters, and network throughput — without ever logging in.

1.2 Root Cause

The global before_request handler in gui_dashboard/app.py whitelisted the debug API prefix alongside genuinely public routes such as login and signup:

```
if (
    request.path.startswith('/api/debug') # <-- mistake
    or request.path.startswith('/api/auth/')
    or request.path.startswith('/socket.io')
    or request.endpoint in ('hero', 'demo', 'login', 'signup',
        'static', 'api_username_availability')
    or request.path.startswith('/static/')
):
    return None
if not session.get('user_id'):
    return redirect(url_for('login', next=request.path))
```

1.3 Steps to Reproduce

- Start FalconStrix and confirm it is reachable on the network (http://192.168.30.1:5001 in the test lab).
- Using Burp Repeater, send a GET request to /api/debug/events with no Cookie header (fully anonymous client).
- Observe the response.

Request:

```
GET /api/debug/events HTTP/1.1
Host: 192.168.30.1:5001
[No Cookie header sent]
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 21172

{
  "events": [
    {
      "event_type": "dashboard_snapshot_emit",
      "payload": {
        "active_alerts": 0, "ipc_total": 2, "kill_cnt": 2,
        "recent_events": 8, "state": "NORMAL"
      },
      "ts": "2026-07-22T16:31:26"
    },
    ... (70+ additional entries returned per request)
  ],
  "ok": true
}
```

Control test: the same technique against a protected route not on the whitelist (/dashboard) correctly returned 302 Found to /login, confirming the auth mechanism itself works and the debug routes were a specific, isolated oversight rather than a systemic failure.

1.4 Impact

- Unauthenticated disclosure of internal SOC telemetry (state, alert counts, network volume).
- Reconnaissance value: an attacker can observe whether the monitoring system currently believes it is under attack, enabling better-timed follow-on actions.
- Notable irony: the disclosure occurs in a security-monitoring product itself.
- No code execution, credential theft, or data modification demonstrated; impact limited to information disclosure.

1.5 Remediation

Control 1 – remove debug routes from the public whitelist:

```
if (
  request.path.startswith('/api/auth/')
  or request.path.startswith('/socket.io')
  or request.endpoint in ('hero', 'demo', 'login', 'signup',
    'static', 'api_username_availability')
  or request.path.startswith('/static/')
):
  return None
if not session.get('user_id'):
  return redirect(url_for('login', next=request.path))
```

Control 2 – defense-in-depth admin check inside the routes:

```
@app.route('/api/debug/events')
def api_debug_events():
  if (session.get('role') or '').lower() != 'admin':
    return jsonify({'ok': False, 'error': 'forbidden'}), 403
```

```

...

@app.route('/api/debug/last-snapshot')
def api_debug_last_snapshot():
    if (session.get('role') or '').lower() != 'admin':
        return jsonify({'ok': False, 'error': 'forbidden'}), 403
    ...

```

1.6 Verification (Post-Fix Re-Testing)

Test A – fully anonymous request:

```

GET /api/debug/events HTTP/1.1
[No Cookie header]

--- Response ---
HTTP/1.1 302 FOUND
Location: /login?next=/api/debug/events

```

Test B – authenticated as a non-admin user:

```

GET /api/debug/events HTTP/1.1
Cookie: session=<valid session, non-admin account>

--- Response ---
HTTP/1.1 403 FORBIDDEN
{"error": "forbidden", "ok": false}

```

Test Condition	Before Fix	After Fix
No session cookie (anonymous)	200 OK – data disclosed	302 Found – redirect to /login
Valid session, non-admin role	200 OK – data disclosed	403 Forbidden – access denied
Valid session, admin role	200 OK – data disclosed	200 OK – data returned (expected)

Both tests were repeated against `/api/debug/last-snapshot` with identical results.

Finding 2 — Auth Lockout Self-Bypass & Missing Authorization on Unlock (CWE-287 / CWE-863)

Report ID	FALSTRX-2026-003
Affected Components	backend/fsm_service.py (maybe_unlock_locked_state); gui_dashboard/static/script.js (auto-reeval)
Vulnerability Class	Improper Authentication (CWE-287) – lockout does not persist as intended; Incorrect Authoriz
Severity	Medium — ineffective account-lockout control, compounded by a broken access control gap
Status	Fixed & Verified — 22–23 July 2026

2.1 Summary

FalconStrix implements a failed-login lockout: after 3 failed attempts from an IP, the FSM (Finite State Machine) security state escalates to LOCKED and the UI displays a full-screen “THREAT LEVEL: CRITICAL” overlay. In practice this lockout provided little real protection: the LOCKED state cleared itself automatically within seconds of the next page load, and the endpoint responsible for clearing it could be called by any authenticated user — not only admins, despite code comments describing it as an admin-only workflow.

2.2 Root Cause

Three contributing issues were identified in combination:

(a) Frontend auto-triggers an unlock check on every LOCKED page load:

```
// gui_dashboard/static/script.js
if (activeCount === 0 && !lockAutoReevalInFlight && !lockAutoReevalUnsupported) {
  fetch('/api/fsm/reevaluate', { method: 'POST', ... })
  .then((r) => r.json())
  .then((payload) => {
    if (payload && payload.snapshot) {
      applyGlobalStateUi(payload.snapshot.state);
    }
  });
}
```

(b) The unlock logic only checks unresolved alert count, not lock cause:

```
# backend/fsm_service.py (before fix)
def maybe_unlock_locked_state(actor_username='unknown'):
    """
    Admin-only workflow uses this after resolving alerts:
    LOCKED -> NORMAL only when there are no unresolved alerts remaining.
    """
    unresolved_q = "SELECT COUNT(*) AS cnt FROM Alerts WHERE is_resolved = FALSE"
    unresolved = fetch_query(unresolved_q, fetchall=False)
    unresolved_cnt = int(unresolved['cnt']) if unresolved and unresolved.get('cnt') is not None else 0
    curr = get_current_state()
    if curr != 'LOCKED':
        return {'changed': False, 'reason': f'FSM is {curr}, no lock-clear needed'}
    if unresolved_cnt > 0:
```

```
return {'changed': False, 'reason': f'{unresolved_cnt} unresolved alert(s) remain'}
reason = f"Admin '{actor_username}' resolved active cases and cleared LOCKED state"
update_fsm_state('NORMAL', reason, trigger_active_defense=False)
return {'changed': True, 'reason': reason}
```

Because an authentication lockout (unlike an alert-driven lockout) has zero unresolved alerts by definition, this function always concluded it was safe to auto-unlock — regardless of the actual lock reason.

(c) The route calling this logic had no role check, despite its docstring's claim:

```
# gui_dashboard/app.py (before fix)
@app.route('/api/fsm/reevaluate', methods=['POST'])
def api_fsm_reevaluate():
    """
    Force an FSM recalculation and unlock check.
    Used by LOCKED overlay when counts are shown as zero.
    """
    from fsm_service import tick_fsm, maybe_unlock_locked_state
    actor = session.get('username') or 'system'
    tick_fsm()
    maybe_unlock_locked_state(actor_username=actor) # no role check
    ...
```

The separate `maybe_unlock_locked_state` docstring calls this an “Admin-only workflow,” but nothing in either function enforced that — any authenticated user's browser silently triggered it every time the LOCKED overlay rendered.

Separately, it was confirmed that the per-IP failed-login counter (`login_failure_by_ip`, in-memory, keyed by `request.remote_addr`) is cleared on **any** successful login from that IP (`clear_login_failures(request.remote_addr)`), regardless of which account logged in. On a shared IP (NAT, office network, shared machine), one user's successful login resets the failed-attempt count for everyone on that IP.

2.3 Steps to Reproduce

- Submit 3 incorrect password attempts against `/login` to trigger the lockout. Confirm the “THREAT LEVEL: CRITICAL — State: LOCKED” overlay appears, with a lock reason such as: *“Dashboard auth: 3 failed attempts (IP 192.168.1.17, user 'sf') — FSM LOCKED”*.
- Reload the dashboard (or simply wait a few seconds while the overlay is displayed) without taking any explicit unlock action.
- Observe: the LOCKED overlay clears itself automatically within seconds, and the state badge returns to NORMAL.
- Separately, log in with a valid, non-admin session and send `POST /api/fsm/reevaluate` directly via Burp while the FSM is LOCKED with zero unresolved alerts — observe it succeeds and clears the lockout, despite the account not being an admin.

Evidence – lockout state immediately before auto-clear:

```
THREAT LEVEL: CRITICAL
DEFENSIVE MODE ACTIVATED
```

```
No active alerts detected. Auto re-checking FSM now to return to NORMAL.
```

```
State: LOCKED
Open Alerts: 0
Critical / High / Medium: 0 / 0 / 0
Lock reason (backend/DB): Dashboard auth: 3 failed attempts (IP 192.168.1.17, user 'sf') – FSM LOCKED
```

Within seconds of this screen rendering, the frontend's automatic `fetch('/api/fsm/reevaluate', ...)` call resolved the state back to NORMAL without any user or admin action.

2.4 Impact

- The failed-login lockout provided minimal real protection: the LOCKED status was cosmetic and self-cleared almost immediately, meaning a scripted brute-force attempt would rarely, if ever, be meaningfully slowed by it.
- Any authenticated user — not just an admin — could trigger the unlock action, directly contradicting the intended admin-only control described in the code's own comments.
- The per-IP failure counter being cleared by any successful login on that IP further weakens the control on shared/NAT'd networks, where one legitimate login resets the attempt budget for anyone else on the same address.
- Combined effect: an attacker attempting to brute-force a login was, in practice, only briefly inconvenienced rather than blocked.

2.5 Remediation

Control 1 – make the unlock logic reason-aware (fsm_service.py):

```
def get_last_lock_reason():
    query = (
        "SELECT reason FROM FSM_State_History "
        "WHERE new_state = 'LOCKED' "
        "ORDER BY changed_at DESC LIMIT 1"
    )
    res = fetch_query(query, fetchall=False)
    if res:
        return res['reason']
    return ''

def maybe_unlock_locked_state(actor_username='unknown'):
    """
    Admin-only workflow uses this after resolving alerts:
    LOCKED -> NORMAL only when there are no unresolved alerts remaining.
    """
    curr = get_current_state()
    if curr != 'LOCKED':
        return {'changed': False, 'reason': f'FSM is {curr}, no lock-clear needed'}

    last_reason = get_last_lock_reason()
    if 'Dashboard auth:' in last_reason:
        return {'changed': False, 'reason': 'Auth lockout requires manual admin clear'}

    unresolved_q = "SELECT COUNT(*) AS cnt FROM Alerts WHERE is_resolved = FALSE"
```

```

unresolved = fetch_query(unresolved_q, fetchall=False)
unresolved_cnt = int(unresolved['cnt']) if unresolved and unresolved.get('cnt') is not None else 0
if unresolved_cnt > 0:
    return {'changed': False, 'reason': f'{unresolved_cnt} unresolved alert(s) remain'}

reason = f"Admin '{actor_username}' resolved active cases and cleared LOCKED state"
update_fsm_state('NORMAL', reason, trigger_active_defense=False)
return {'changed': True, 'reason': reason}

```

This ensures a lockout whose most recent cause was a dashboard authentication failure can no longer be cleared by the automatic, unauthenticated-by-role reevaluate path — regardless of alert counts.

Control 2 – explicit, admin-gated manual clear path (fsm_service.py):

```

def clear_auth_lockout(actor_username='unknown'):
    """
    Explicit, admin-only action to clear a LOCKED state caused by a
    dashboard authentication lockout. Unlike maybe_unlock_locked_state(),
    this does not check alert counts or lock reason – it's a deliberate
    override, meant to be called only from an admin-gated route.
    """
    curr = get_current_state()
    if curr != 'LOCKED':
        return {'changed': False, 'reason': f'FSM is {curr}, no lock-clear needed'}

    reason = f"Admin '{actor_username}' manually cleared auth lockout"
    update_fsm_state('NORMAL', reason, trigger_active_defense=False)
    return {'changed': True, 'reason': reason}

```

Control 3 – admin-gated route enforcing the role check (app.py):

```

@app.route('/api/fsm/clear-auth-lockout', methods=['POST'])
def api_fsm_clear_auth_lockout():
    """Admin-only: manually clear a LOCKED state caused by a dashboard auth lockout."""
    if (session.get('role') or '').lower() != 'admin':
        return jsonify({'ok': False, 'error': 'forbidden'}), 403
    try:
        from fsm_service import clear_auth_lockout
        result = clear_auth_lockout(actor_username=session.get('username'))
        st = get_effective_fsm_state()
        snap = build_dashboard_snapshot(st)
        return jsonify({'ok': True, 'result': result, 'state': st, 'snapshot': snap})
    except Exception as e:
        return jsonify({'ok': False, 'error': str(e)}), 500

```

A corresponding “Clear Auth Lockout (Admin)” button was added to the LOCKED overlay UI, hidden by default and shown only when the logged-in user's role is admin (via `window.FALCON_USER_ROLE`), giving admins a real, intentional way to clear an auth lockout now that the automatic path correctly refuses to.

2.6 Verification (Post-Fix Re-Testing)

After remediation, an auth lockout was triggered again (3 failed logins). The LOCKED overlay was observed to **persist** across page reloads and the automatic reevaluate call, instead of clearing itself within seconds as before. Live Alerts correctly showed 0 tracked alerts, confirming the persistence was due to the new auth-lockout check, not leftover unresolved alerts. Logging in as a second user ('zaid') while the lockout was active showed the same

system-wide STATE: LOCKED badge, confirming the lock is a genuine FSM-level state rather than tied to a single session.

Test Condition	Before Fix	After Fix
LOCKED overlay left on screen, page reloaded	Auto-clears to NORMAL within seconds	Remains LOCKED indefinitely
Non-admin session calls /api/fsm/reevaluate	Can clear the lockout	Lockout persists (reason-aware check blocks it)
Admin clicks new "Clear Auth Lockout" button	UI control did not exist	Lockout cleared via dedicated admin-gated route
Non-admin session calls /api/fsm/clear-auth-lockout	API control did not exist	403 Forbidden

Note: the direct-403 row for the new admin-only route follows from the same session-role-check pattern independently verified for Finding 1's debug routes; it was not re-screenshotted separately for this finding.

2.7 Related Observation (Not Independently Remediated)

The per-IP failed-login counter being reset by *any* successful login from that IP (clear_login_failures(request.remote_addr) in the login route) was identified during testing but is documented here as a related observation rather than a separately remediated finding. On shared/NAT'd networks this could allow a failed-attempt budget to be effectively refreshed by unrelated legitimate logins. Recommended follow-up: scope the failed-attempt counter to the specific username being attempted, or a combination of IP and username, rather than IP alone.

Testing Environment

- **Target:** FalconStrix run locally on a Windows 11 host, Flask bound to 0.0.0.0:5001, reachable only via an isolated VMware virtual network (Host-only / NAT adapters, no public exposure).
- **Attacker:** Kali Linux in VMware, using Burp Suite Community Edition as an intercepting proxy, with Burp's CA certificate imported into the attacking browser for HTTPS interception of third-party CDN traffic (unrelated to the target application's own HTTP traffic).
- All testing was performed exclusively against a locally-hosted instance of the researcher's own project. No third-party systems, production services, or other individuals' applications were tested at any point.

This report documents self-identified findings in a personal project, produced for educational / responsible-disclosure practice purposes.